

Pandas_and_Numpy_Task-Examples

July 5, 2026

1 Name: Anjali Suresh Chopade

2 Roll_no.: 2013

3 _____

4 NumPy Example

5 _____

5.1 1. The ndarray and why NumPy is memory-efficient ?

```
[1]: import numpy as np

arr = np.array([10,20,30,40,50])
print(arr)
```

```
[10 20 30 40 50]
```

5.2 2. Creating arrays (array , arange, linspace, zeros, ones, random.randint)
?

1. array() : Creates a array fro a list or tuple.

```
[2]: # 1. array() : Creates a array fro a list or tuple.

import numpy as np

a = np.array([1,2,3,4,5])
print(a)
```

```
[1 2 3 4 5]
```

```
[3]: # 2. arange(): Creates an array with values in a given range.

import numpy as np

b = np.arange(1,11)
print(b)
```

```
[ 1  2  3  4  5  6  7  8  9 10]
```

```
[4]: # 3. linspace():Creates equally spaced values between two numbers.
```

```
import numpy as np

c = np.linspace(2,20,5)
print(c)
```

```
[ 2.   6.5 11.  15.5 20. ]
```

```
[5]: # 4. zeros(): Creates an array filled with zeros.
```

```
import numpy as np

d = np.zeros(10)
print(d)
```

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

```
[6]: # 5. ones(): Creates an array filled with ones.
```

```
import numpy as np

e = np.ones(5)
print(e)
```

```
[1. 1. 1. 1. 1.]
```

```
[7]: # 6. random.randint(): Creates an array of random integers.
```

```
import numpy as np

a = np.random.randint(1, 10, 5)
print(a)
```

```
[6 3 5 1 3]
```

5.2.1 3. Array attributes (ndim, shape, size, dtype, itemsize) ?

```
[8]: # 1. ndim: Returns the number of dimensions of an array.
```

```
import numpy as np

a = np.array([[1,2],[3,4]])
print(a.ndim)
```

[9]: *# 2. shape: Returns the shape (rows and columns) of an array.*

```
import numpy as np

a = np.array([[10, 20], [30, 40]])
print(a.shape)
```

(2, 2)

[10]: *# 3. size: Returns the total number of elements in an array.*

```
import numpy as np

a = np.array([10, 20, 30, 40])
print(a.size)
```

4

[11]: *# 4. dtype: Returns the data type of the array elements.*

```
import numpy as np

a = np.array([11, 22, 33, 44, 55])
print(a.dtype)
```

int64

[12]: *# 5. itemsize: Returns the size of each element in bytes.*

```
import numpy as np

a = np.array([120, 130, 140, 150, 160, 170])
print(a.itemsize)
```

8

5.3 4. Indexing - Single Value

[15]: *# 1D array*

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print(arr[2])
```

30

[16]: *# 2D array*

```
import numpy as np
```

```
arr = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])

print(arr[1, 2])
```

60

```
[17]: # Negative indexing
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print(arr[-1])
```

50

```
[19]: # 3D Array
import numpy as np

arr = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
])

print(arr[1, 0, 1])
```

6

5.4 5. Slicing - Multiple values

```
[20]: import numpy as np

arr = np.array(['Banana', 'Apple', 'Mango', 'Orange'])

# index 1 to 2 (stop not included)
print(arr[1:3])
```

['Apple' 'Mango']

```
[21]: # start to index 1
print(arr[:2])
```

['Banana' 'Apple']

```
[22]: # index 2 to end
print(arr[2:])
```

```
['Mango' 'Orange']
```

```
[23]: # every second element (step = 2)
      print(arr[::2])
```

```
['Banana' 'Mango']
```

```
[24]: # negative slicing (from end side)
      print(arr[-3:-1])
```

```
['Apple' 'Mango']
```

5.5 6. Fancy indexing

```
[25]: import numpy as np

      arr = np.array([10, 20, 30, 40, 50, 60])

      # selecting elements at index 0, 2, and 4
      print(arr[[0, 2, 4]])
```

```
[10 30 50]
```

```
[26]: import numpy as np

      arr = np.array([
          [10, 20, 30],
          [40, 50, 60],
          [70, 80, 90]
      ])

      # selecting row 0 and row 2
      print(arr[[0, 2]])
```

```
[[10 20 30]
 [70 80 90]]
```

```
[27]: # selecting (0,1)=20 and (1,2)=60
      print(arr[[0, 1], [1, 2]])
```

```
[20 60]
```

5.6 7. Boolean (Mask) Indexing / filtering

```
[28]: import numpy as np

      arr = np.array([10, 20, 30, 40, 50, 60])

      # selecting values greater than 30
      print(arr[arr > 30])
```

[40 50 60]

```
[29]: arr > 30
```

```
[29]: array([False, False, False,  True,  True,  True])
```

```
[30]: # creating a boolean condition (mask)
      mask = arr > 30

      # applying mask to filter values
      print(arr[mask])
```

[40 50 60]

```
[31]: # selecting values greater than 20 and less than 60
      print(arr[(arr > 20) & (arr < 60)])
```

[30 40 50]

5.7 8. Mutabiliti - Replace,Append,Delete

```
[32]: # replace() - replace the value
      import numpy as np

      arr = np.array([10, 20, 30, 40])

      # replacing value at index 1
      arr[1] = 99

      print(arr)
```

[10 99 30 40]

```
[33]: arr[1] = 99
```

```
[35]: # append() - add the new element at the end
      import numpy as np

      arr = np.array([10, 20, 30])

      # adding new element at end
      arr = np.append(arr, 40)

      print(arr)
```

[10 20 30 40]

```
[36]: # delete() - delete the element
      import numpy as np
```

```
arr = np.array([10, 20, 30, 40])

# deleting element at index 2
arr = np.delete(arr, 2)

print(arr)
```

[10 20 40]

5.8 9. Sorting

```
[37]: import numpy as np

arr = np.array([40, 10, 30, 20, 50])

# sorting in ascending order
print(np.sort(arr))
```

[10 20 30 40 50]

```
[38]: np.sort(arr)
```

```
[38]: array([10, 20, 30, 40, 50])
```

```
[39]: # sorting in descending order
print(np.sort(arr)[::-1])
```

[50 40 30 20 10]

```
[40]: arr = np.array([
    [40, 10, 30],
    [20, 50, 60]
])

# sorting each row
print(np.sort(arr))
```

[[10 30 40]
 [20 50 60]]

```
[41]: # sorting column-wise (axis = 0)
print(np.sort(arr, axis=0))
```

[[20 10 30]
 [40 50 60]]

5.9 10. Copy vs Reference in NumPy (.copy)

```
[44]: # 1. Reference (No .copy())
import numpy as np

arr1 = np.array([10, 20, 30, 40])

# creating reference (same memory location)
arr2 = arr1

# modifying arr2
arr2[0] = 99

print("arr1:", arr1)
print("arr2:", arr2)
```

```
arr1: [99 20 30 40]
arr2: [99 20 30 40]
```

```
[46]: # True Copy (.copy())
import numpy as np

arr1 = np.array([10, 20, 30, 40])

# creating an independent copy
arr2 = arr1.copy()

# modifying arr2
arr2[0] = 99

print("arr1:", arr1)
print("arr2:", arr2)
```

```
arr1: [10 20 30 40]
arr2: [99 20 30 40]
```

5.10 11. Rounding (np.around)

```
[47]: import numpy as np

arr = np.array([1.234, 2.567, 3.891])

# rounding to nearest integer
print(np.around(arr))
```

```
[1.  3.  4.]
```

```
[48]: np.around(arr)
```

```
[48]: array([1., 3., 4.])
```

```
[49]: # rounding to 2 decimal places  
print(np.around(arr, 2))
```

```
[1.23 2.57 3.89]
```

```
[50]: np.around(arr, 2)
```

```
[50]: array([1.23, 2.57, 3.89])
```

```
[51]: print(np.around(arr, 1))
```

```
[1.2 2.6 3.9]
```

5.11 12. Transpose

```
[52]: import numpy as np  
  
arr = np.array([  
    [1, 2, 3],  
    [4, 5, 6]  
)  
  
# transpose of array  
print(arr.T)
```

```
[[1 4]  
 [2 5]  
 [3 6]]
```

```
[53]: print(np.transpose(arr))
```

```
[[1 4]  
 [2 5]  
 [3 6]]
```

```
[54]: np.transpose(arr)
```

```
[54]: array([[1, 4],  
            [2, 5],  
            [3, 6]])
```

```
[55]: arr = np.array([  
    [1, 2],  
    [3, 4]  
)  
  
print(arr.T)
```

```
[[1 3]
 [2 4]]
```

```
[56]: arr.T
```

```
[56]: array([[1, 3],
           [2, 4]])
```

5.12 13. Aggregation (np.sum and friends)

```
[57]: # sum (np.sum)
import numpy as np

arr = np.array([10, 20, 30, 40])

# sum of all elements
print(np.sum(arr))
```

```
100
```

```
[58]: np.sum(arr)
```

```
[58]: np.int64(100)
```

```
[59]: # mean: average of elements
print(np.mean(arr))
```

```
25.0
```

```
[61]: # Max: maximum value
print(np.max(arr))
```

```
40
```

```
[62]: # min : minimum value
print(np.min(arr))
```

```
10
```

```
[63]: # Standard Deviation: spread of data
print(np.std(arr))
```

```
11.180339887498949
```

5.13 14. Reshaping, Flatten vs Ravel

```
[64]: # Reshaping (reshape): Changes the shape of the array (data size remains same)
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])
```

```
# reshaping into 2 rows and 3 columns
new_arr = arr.reshape(2, 3)

print(new_arr)
```

```
[[1 2 3]
 [4 5 6]]
```

```
[67]: # Flatten: Converts array into 1D array (copy)
arr = np.array([[1, 2, 3],
                [4, 5, 6]])

flat = arr.flatten()

print(flat)
```

```
[1 2 3 4 5 6]
```

```
[68]: # Ravel: Converts array into 1D array (view)
rav = arr.ravel()

print(rav)
```

```
[1 2 3 4 5 6]
```

5.14 15. Broadcastig

```
[69]: import numpy as np

arr = np.array([1, 2, 3, 4])

# scalar 5 is broadcasted to all elements
print(arr + 5)
```

```
[6 7 8 9]
```

```
[70]: a = np.array([[1, 2, 3],
                   [4, 5, 6]])

b = np.array([10, 20, 30])

print(a + b)
```

```
[[11 22 33]
 [14 25 36]]
```

```
[71]: a = np.array([[1],
                    [2],
                    [3]])
```

```
b = np.array([10, 20, 30])  
  
print(a + b)
```

```
[[11 21 31]  
 [12 22 32]  
 [13 23 33]]
```

5.15 16. Vectorization

```
[72]: import numpy as np  
  
arr = np.array([1, 2, 3, 4])  
  
result = []  
  
# using loop (slow method)  
for i in arr:  
    result.append(i * 2)  
  
print(result)
```

```
[np.int64(2), np.int64(4), np.int64(6), np.int64(8)]
```

```
[73]: import numpy as np  
  
arr = np.array([1, 2, 3, 4])  
  
# vectorized operation (fast)  
result = arr * 2  
  
print(result)
```

```
[2 4 6 8]
```

```
[74]: a = np.array([10, 20, 30])  
      b = np.array([1, 2, 3])  
  
# vectorized addition  
print(a + b)
```

```
[11 22 33]
```

```
[75]: arr = np.array([10, 15, 20, 25, 30])  
  
# vectorized filtering  
print(arr[arr > 20])
```

```
[25 30]
```

5.16 17. Universal Functions(ufuncs)

```
[76]: for i in range(len(a)):
      c[i] = a[i] + b[i]
```

```
[77]: import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Addition
print(np.add(a, b))      # [5 7 9]

# Subtraction
print(np.subtract(a, b)) # [-3 -3 -3]

# Multiplication
print(np.multiply(a, b)) # [4 10 18]

# Division
print(np.divide(a, b))   # [0.25 0.4 0.5]
```

```
[5 7 9]
[-3 -3 -3]
[ 4 10 18]
[0.25 0.4 0.5 ]
```

```
[78]: print(np.power(a, 2))    # square each element → [1 4 9]
      print(np.mod(b, a))     # remainder → [0 1 0]
```

```
[1 4 9]
[0 1 0]
```

```
[79]: a = np.array([10, 20, 30])
      b = np.array([15, 20, 25])

print(np.greater(a, b))    # [False False True]
print(np.equal(a, b))      # [False True False]
```

```
[False False True]
[False True False]
```

```
[80]: angles = np.array([0, np.pi/2, np.pi])

print(np.sin(angles))    # sine values
print(np.cos(angles))    # cosine values
```

```
[0.0000000e+00 1.0000000e+00 1.2246468e-16]
[ 1.000000e+00  6.123234e-17 -1.000000e+00]
```

5.17 18. Axis parameter explained visually

```
[83]: np.sum(a, axis=0)
```

```
[83]: array([5, 7, 9])
```

```
[84]: np.sum(a, axis=1)
```

```
[84]: array([ 6, 15])
```

5.18 19. Stacking and Splitting

```
[85]: import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Stack arrays vertically
result = np.vstack((a, b))

print(result)
```

```
[[1 2 3]
 [4 5 6]]
```

```
[86]: import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Stack arrays horizontally
result = np.hstack((a, b))

print(result)
```

```
[1 2 3 4 5 6]
```

```
[87]: import numpy as np

a = np.array([1, 2, 3, 4, 5, 6])

# Split into 3 equal parts
result = np.split(a, 3)

print(result)
```

```
[array([1, 2]), array([3, 4]), array([5, 6])]
```

5.19 20. Linear Algebra with Numpy

```
[88]: import numpy as np

A = np.array([[1, 2],
              [3, 4]])

B = np.array([[5, 6],
              [7, 8]])

# Matrix multiplication
result = np.dot(A, B)

print(result)
```

```
[[19 22]
 [43 50]]
```

```
[89]: import numpy as np

A = np.array([[1, 2],
              [3, 4]])

# Transpose
print(A.T)
```

```
[[1 3]
 [2 4]]
```

```
[90]: import numpy as np

A = np.array([[1, 2],
              [3, 4]])

# Determinant
print(np.linalg.det(A))
```

```
-2.0000000000000004
```

```
[91]: import numpy as np

A = np.array([[1, 2],
              [3, 4]])

# Inverse
print(np.linalg.inv(A))
```

```
[[ -2.   1. ]
 [ 1.5 -0.5]]
```

5.20 21. Statistics with Numpy

```
[92]: import numpy as np

arr = np.array([10, 20, 30, 40, 50])

# Calculate mean
print(np.mean(arr))
```

30.0

```
[93]: import numpy as np

arr = np.array([10, 20, 30, 40, 50])

# Calculate median
print(np.median(arr))
```

30.0

```
[94]: import numpy as np

arr = np.array([10, 20, 30, 40, 50])

# Standard deviation
print(np.std(arr))
```

14.142135623730951

```
[95]: import numpy as np

arr = np.array([10, 20, 30, 40, 50])

# Variance
print(np.var(arr))
```

200.0

5.21 22. Matrix Operations

```
[101]: # Matrix addition
print(A + B)

# OR
print(np.add(A, B))
```

```
[[ 6  8]
 [10 12]]
[[ 6  8]
 [10 12]]
```

```
[102]: # Matrix subtraction
print(A - B)

# OR
print(np.subtract(A, B))
```

```
[[ -4 -4]
 [ -4 -4]]
[[ -4 -4]
 [ -4 -4]]
```

```
[103]: # Element-wise multiplication
print(A * B)

# OR
print(np.multiply(A, B))
```

```
[[ 5 12]
 [21 32]]
[[ 5 12]
 [21 32]]
```

```
[104]: # Matrix multiplication
print(A @ B)

# OR
print(np.dot(A, B))
```

```
[[19 22]
 [43 50]]
[[19 22]
 [43 50]]
```

```
[105]: # Matrix division
print(A / B)
```

```
[[0.2      0.33333333]
 [0.42857143 0.5      ]]
```

```
[106]: # Matrix transpose
print(A.T)
```

```
[[1 3]
 [2 4]]
```

```
[107]: # Square of matrix
print(np.linalg.matrix_power(A, 2))
```

```
[[ 7 10]
 [15 22]]
```

```
[108]: # Matrix inverse
print(np.linalg.inv(A))
```

```
[[ -2.   1. ]
 [ 1.5 -0.5]]
```

```
[109]: # Determinant
print(np.linalg.det(A))
```

```
-2.0000000000000004
```

```
[110]: # Trace
print(np.trace(A))
```

```
5
```

```
[111]: arr = np.array([1, 2, 3, 4, 5, 6])
```

```
# Reshape into 2x3 matrix
print(arr.reshape(2, 3))
```

```
[[1 2 3]
 [4 5 6]]
```

```
[112]: # Shape
print(A.shape)
```

```
(2, 2)
```

5.22 23. One-Page Cheat Sheet

```
[113]: np.array([1,2,3])
np.zeros((2,3))
np.ones((2,3))
np.eye(3)
np.arange(0,10,2)
np.linspace(0,1,5)
np.random.rand(2,3)
np.random.randint(1,10,(2,2))
```

```
[113]: array([[3, 2],
             [1, 7]], dtype=int32)
```

```
[114]: arr.ndim      # Number of dimensions
arr.shape           # Shape
arr.size            # Total elements
arr.dtype           # Data type
arr.itemsize        # Size of one element (bytes)
```

```
[114]: 8
```

```
[118]: arr[0]
      arr[-1]
      arr[1:5]
      arr[:1]
      arr[1:]
      arr[0:2:1]
```

```
[118]: array([1, 2])
```

```
[119]: arr[arr>5]
      arr[[0,2,4]]
```

```
[119]: array([1, 3, 5])
```

```
[120]: arr.reshape(2,3)
      arr.flatten()
      arr.ravel()
      arr.T
```

```
[120]: array([1, 2, 3, 4, 5, 6])
```

6 _____

7 Pandas Example

8 _____

9 Section 1: Conceptual Foundations

```
[3]: import numpy as np
      import pandas as pd

      print("Pandas version:", pd.__version__)
      print("NumPy version:", np.__version__)
```

Pandas version: 2.3.1

NumPy version: 2.4.6

```
[4]: # Quick illustration: same data, four containers
      data = ['Banana', 5, 4.3]

      py_list = data
      py_dict = {'item': 'banana', 'count': 5, 'price': 4.3}
      np_arr = np.array(data, dtype=object)
      pd_series = pd.Series(data, index=['item', 'count', 'price'])

      print("List:", py_list)
```

```

print("Dict:", py_dict)
print("NumPy array:", np_arr)
print("\nPandas Series (labeled, mixed types):")
print(pd_series)

```

```

List: ['Banana', 5, 4.3]
Dict: {'item': 'banana', 'count': 5, 'price': 4.3}
NumPy array: ['Banana' 5 4.3]

```

```

Pandas Series (labeled, mixed types):
item      Banana
count          5
price        4.3
dtype: object

```

10 Section 2: Pandas Series

```

[5]: # Create a Series from a list (default integer index)
lst = ['Banana', 5]
s = pd.Series(lst)
print(s)
print(type(s))

```

```

0    Banana
1         5
dtype: object
<class 'pandas.core.series.Series'>

```

```

[6]: # Create a Series with a custom index
lst = ['Banana', 5]
s = pd.Series(lst, index=['a', 'c'])
print(s)

```

```

a    Banana
c         5
dtype: object

```

```

[7]: # Create a Series with a custom index
lst = ['Banana', 5]
s = pd.Series(lst, index=['a', 'c'])
print(s)

```

```

a    Banana
c         5
dtype: object

```

```

[8]: # Create a Series from a NumPy array
a = np.array(['Apple', 50])

```

```
print(a, type(a))

s = pd.Series(a, index=['a', 'b'])
print(s)

['Apple' '50'] <class 'numpy.ndarray'>
a      Apple
b         50
dtype: object
```

```
[9]: # Create a Series from a dictionary -> keys become the index
d = {'a': [10, 20, 30], 'b': [30, 20, 10]}
s = pd.Series(d)
print(s)
```

```
a      [10, 20, 30]
b      [30, 20, 10]
dtype: object
```

```
[10]: # Indexing into a Series
s = pd.Series([1, 2, 3, 4],
index=['a', 'n', 's', 'h'])
print("By label:", s['a'])
print("By position:", s.iloc[1])
print("Slice by label:\n", s['n':'h'])          # label slicing is INCLUSIVE
↪ print("Slice by position:\n", s.iloc[1:3])
# positional slicing is EXCLUSIVE of end
```

```
By label: 1
By position: 2
Slice by label:
n      2
s      3
h      4
dtype: int64
```

11 Section 3: DataFrame Creation

```
[11]: # Create DataFrame from a dictionary
s = {'coll': [10, 20]}
df_dict = pd.DataFrame(s)
print(df_dict)
print(type(df_dict))
print("Shape:", df_dict.shape)
```

```
coll
0    10
1    20
```

```
<class 'pandas.core.frame.DataFrame'>  
Shape: (2, 1)
```

```
[12]: # Create DataFrame from a dictionary  
s = {'coll': [11, 22]}  
df_dict = pd.DataFrame(s)  
print(df_dict)  
print(type(df_dict))  
print("Shape:", df_dict.shape)
```

```
   coll  
0     11  
1     22  
<class 'pandas.core.frame.DataFrame'>  
Shape: (2, 1)
```

```
[13]: # Create DataFrame from a dictionary of equal-length lists (most common pattern)  
df_multi = pd.DataFrame({  
    'name': ['Alisha', 'Rohan', 'Mahi'],  
    'age'  : [20, 21, 11],  
    'score': [8.5, 2.0, 7.3]})  
df_multi
```

```
[13]:
```

	name	age	score
0	Alisha	20	8.5
1	Rohan	21	2.0
2	Mahi	11	7.3

12 Section 4: Combining DataFrames

```
[14]: temp_df = pd.DataFrame({  
    'city': ['Pune', 'Nagpur', 'Nashik', 'Kolhapur'],  
    'temp': [30, 38, 29, 31]  
})  
  
hum_df = pd.DataFrame({  
    'city': ['Pune', 'Nagpur', 'Aurangabad', 'Solapur'],  
    'humidity': [72, 58, 65, 60]  
})  
  
print(temp_df)  
print()  
print(hum_df)
```

```
   city  temp  
0   Pune    30  
1 Nagpur    38  
2 Nashik    29
```

```
3 Kolhapur 31
```

	city	humidity
0	Pune	72
1	Nagpur	58
2	Aurangabad	65
3	Solapur	60

```
[15]: # concat: stacks rows on top of each other (note duplicate index unless
      ↪ ignore_index=True)
      pd.concat([temp_df, hum_df],
      ignore_index=True)
```

```
[15]:
```

	city	temp	humidity
0	Pune	30.0	NaN
1	Nagpur	38.0	NaN
2	Nashik	29.0	NaN
3	Kolhapur	31.0	NaN
4	Pune	NaN	72.0
5	Nagpur	NaN	58.0
6	Aurangabad	NaN	65.0
7	Solapur	NaN	60.0

```
[16]: # concat along columns (axis=1) - aligns by index position
      pd.concat([temp_df, hum_df], axis=1)
```

```
[16]:
```

	city	temp	city	humidity
0	Pune	30	Pune	72
1	Nagpur	38	Nagpur	58
2	Nashik	29	Aurangabad	65
3	Kolhapur	31	Solapur	60

```
[17]: # merge: outer join keeps ALL keys from both sides, filling NaN where unmatched
      pd.merge(temp_df, hum_df, on='city', how='outer')
```

```
[17]:
```

	city	temp	humidity
0	Aurangabad	NaN	65.0
1	Kolhapur	31.0	NaN
2	Nagpur	38.0	58.0
3	Nashik	29.0	NaN
4	Pune	30.0	72.0
5	Solapur	NaN	60.0

```
[18]: # merge: inner join keeps ONLY keys present in both DataFrames
      pd.merge(temp_df, hum_df, on='city', how='inner')
```

```
[18]:
```

	city	temp	humidity
0	Pune	30	72

```
1 Nagpur      38      58
```

```
[19]: # merge: left / right joins
print(pd.merge(temp_df, hum_df, on='city', how='left'))
print()
print(pd.merge(temp_df, hum_df, on='city', how='right'))
```

```
      city  temp  humidity
0    Pune    30    72.0
1  Nagpur    38    58.0
2  Nashik    29     NaN
3 Kolhapur    31     NaN
```

```
      city  temp  humidity
0    Pune  30.0    72
1  Nagpur  38.0    58
2 Aurangabad  NaN    65
3  Solapur  NaN    60
```

```
[20]: # join: merges on the INDEX instead of a column
left = temp_df.set_index('city')
right = hum_df.set_index('city')
left.join(right, how='outer')
```

```
[20]:      temp  humidity
city
Aurangabad  NaN    65.0
Kolhapur    31.0     NaN
Nagpur      38.0    58.0
Nashik      29.0     NaN
Pune        30.0    72.0
Solapur     NaN    60.0
```

13 Section 5: Importing / Exporting Data

```
[21]: # --- Build a synthetic student dataset ---

np.random.seed(42)
n = 200

df = pd.DataFrame({
    'Student_ID': np.arange(1, n + 1),
    'Age': np.random.randint(17, 25, n),
    'Gender': np.random.choice(['Male', 'Female'], n, p=[0.55, 0.45]),
    'Maths': np.random.randint(35, 101, n),
    'Science': np.random.randint(35, 101, n),
    'English': np.random.randint(35, 101, n),
```

```

    'Attendance': np.round(np.random.uniform(60, 100, n), 1),
    'Study_Hours': np.round(np.random.uniform(1, 8, n), 1),
    'Sports': np.random.choice(['Yes', 'No'], n, p=[0.4, 0.6]),
    'Result': np.random.choice(['Pass', 'Fail'], n, p=[0.85, 0.15])
})

# Inject a few missing values and duplicate rows
df.loc[[5, 18, 50, 95], 'Study_Hours'] = np.nan
df = pd.concat([df, df.iloc[[10, 10, 25]]], ignore_index=True)

print(df.shape)
print(df.head())

```

(203, 10)

	Student_ID	Age	Gender	Maths	Science	English	Attendance	Study_Hours	\
0	1	23	Male	81	94	54	66.2	2.7	
1	2	20	Female	89	84	46	67.3	1.8	
2	3	21	Male	74	62	81	94.5	3.5	
3	4	23	Male	86	75	35	97.8	3.0	
4	5	19	Female	50	98	48	74.9	3.1	

	Sports	Result
0	Yes	Fail
1	No	Pass
2	No	Fail
3	Yes	Pass
4	No	Pass

[22]: # Export to CSV / Excel / JSON

```

df.to_csv('student_data.csv', index=False)
df.to_excel('student_data.xlsx', index=False)
df.to_json('student_data.json', orient='records')

print("Files written: student_data.csv / .xlsx / .json")

# Read them back

df_csv = pd.read_csv('student_data.csv')
df_xlsx = pd.read_excel('student_data.xlsx')
df_json = pd.read_json('student_data.json')

print(df_csv.shape, df_xlsx.shape, df_json.shape)

df_csv.head(3)

```

Files written: student_data.csv / .xlsx / .json

(203, 10) (203, 10) (203, 10)

```
[22]:
```

	Student_ID	Age	Gender	Maths	Science	English	Attendance	Study_Hours	\
0	1	23	Male	81	94	54	66.2	2.7	
1	2	20	Female	89	84	46	67.3	1.8	
2	3	21	Male	74	62	81	94.5	3.5	

	Sports	Result
0	Yes	Fail
1	No	Pass
2	No	Fail

14 Section 6: DataFrame Attributes

```
[23]: print("shape:", df.shape)
print("size:", df.size)
print("ndim:", df.ndim)
print(df)
```

```
shape: (203, 10)
```

```
size: 2030
```

```
ndim: 2
```

	Student_ID	Age	Gender	Maths	Science	English	Attendance	\
0	1	23	Male	81	94	54	66.2	
1	2	20	Female	89	84	46	67.3	
2	3	21	Male	74	62	81	94.5	
3	4	23	Male	86	75	35	97.8	
4	5	19	Female	50	98	48	74.9	
..	...	...	...	...	...	...		
198	199	21	Female	42	60	72	76.9	
199	200	19	Male	92	53	85	89.5	
200	11	19	Male	53	47	67	88.6	
201	11	19	Male	53	47	67	88.6	
202	26	22	Male	98	45	43	94.2	

	Study_Hours	Sports	Result
0	2.7	Yes	Fail
1	1.8	No	Pass
2	3.5	No	Fail
3	3.0	Yes	Pass
4	3.1	No	Pass
..	...	...	...
198	1.1	No	Fail
199	1.5	Yes	Fail
200	3.7	Yes	Pass
201	3.7	Yes	Pass
202	5.5	Yes	Pass

```
[203 rows x 10 columns]
```

```
[ ]:
```

```
[24]: print("columns:", list(df.columns))
      print("index:", df.index)
```

```
columns: ['Student_ID', 'Age', 'Gender', 'Maths', 'Science', 'English',
'Attendance', 'Study_Hours', 'Sports', 'Result']
index: RangeIndex(start=0, stop=203, step=1)
```

```
[25]: df.values[:5]          # underlying NumPy representation (loses column labels)
```

```
[25]: array([[1, 23, 'Male', 81, 94, 54, 66.2, 2.7, 'Yes', 'Fail'],
            [2, 20, 'Female', 89, 84, 46, 67.3, 1.8, 'No', 'Pass'],
            [3, 21, 'Male', 74, 62, 81, 94.5, 3.5, 'No', 'Fail'],
            [4, 23, 'Male', 86, 75, 35, 97.8, 3.0, 'Yes', 'Pass'],
            [5, 19, 'Female', 50, 98, 48, 74.9, 3.1, 'No', 'Pass']],
      dtype=object)
```

```
[26]: df.axes                # [row index, column index] as a list
```

```
[26]: [RangeIndex(start=0, stop=203, step=1),
      Index(['Student_ID', 'Age', 'Gender', 'Maths', 'Science', 'English',
            'Attendance', 'Study_Hours', 'Sports', 'Result'],
            dtype='object')]
```

```
[27]: df.keys()              # same as df.columns for a DataFrame
```

```
[27]: Index(['Student_ID', 'Age', 'Gender', 'Maths', 'Science', 'English',
            'Attendance', 'Study_Hours', 'Sports', 'Result'],
            dtype='object')
```

```
[28]: df.info()              # dtypes + non-null counts + memory usage - great first_
      ↪ command on any new dataset
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 203 entries, 0 to 202
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Student_ID      203 non-null   int64
1   Age             203 non-null   int32
2   Gender          203 non-null   object
3   Maths           203 non-null   int32
4   Science         203 non-null   int32
5   English         203 non-null   int32
6   Attendance      203 non-null   float64
7   Study_Hours     199 non-null   float64
8   Sports          203 non-null   object
9   Result          203 non-null   object
```

```
dtypes: float64(2), int32(4), int64(1), object(3)
memory usage: 12.8+ KB
```

```
[29]: df.describe()           # summary statistics for numeric columns (count, mean, std, min, quartiles, max)
```

```
[29]:
```

	Student_ID	Age	Maths	Science	English	Attendance	\
count	203.000000	203.000000	203.000000	203.000000	203.000000	203.000000	
mean	99.251232	20.551724	66.477833	67.339901	67.251232	79.928079	
std	58.356373	2.273618	19.036007	20.107575	18.816228	11.853138	
min	1.000000	17.000000	35.000000	35.000000	35.000000	60.400000	
25%	48.500000	19.000000	50.000000	51.000000	51.000000	69.600000	
50%	99.000000	20.000000	66.000000	67.000000	68.000000	80.800000	
75%	149.500000	23.000000	83.000000	85.000000	83.000000	90.000000	
max	200.000000	24.000000	100.000000	100.000000	100.000000	99.900000	

	Study_Hours
count	199.000000
mean	4.621608
std	2.086921
min	1.000000
25%	2.750000
50%	4.800000
75%	6.450000
max	8.000000

```
[30]: df.describe(include='object')           # summary stats for categorical/object columns
```

```
[30]:
```

	Gender	Sports	Result
count	203	203	203
unique	2	2	2
top	Male	No	Pass
freq	109	122	177

15 Section 7: Selecting & Indexing

```
[31]: df.iloc[1]           # first row, by position
```

```
[31]:
```

Student_ID	2
Age	20
Gender	Female
Maths	89
Science	84
English	46
Attendance	67.3
Study_Hours	1.8

```
Sports          No
Result          Pass
Name: 1, dtype: object
```

```
[32]: df.iloc[-1]
```

```
[32]: Student_ID      26
Age                22
Gender            Male
Maths              98
Science            45
English            43
Attendance         94.2
Study_Hours        5.5
Sports            Yes
Result            Pass
Name: 202, dtype: object
```

```
[33]: df.iloc[ [5, 8]]      # specific rows by position
```

```
[33]:   Student_ID  Age Gender  Maths  Science  English  Attendance  Study_Hours  \
5         6    24   Male    47      61      98        70.8         NaN
8         9    23   Male    51      67      45        61.0         7.9

      Sports Result
5      No    Pass
8      No    Pass
```

```
[34]: df.iloc[0:4, 0:1]      # rows 0-2, columns 0-1 (end exclusive, like Python
↳ slicing)
```

```
[34]:   Student_ID
0         1
1         2
2         3
3         4
```

```
[35]: df.loc[0]      # row with index label 0
```

```
[35]: Student_ID      1
Age                23
Gender            Male
Maths              81
Science            94
English            54
Attendance         66.2
Study_Hours        2.7
Sports            Yes
```

```
Result          Fail
Name: 0, dtype: object
```

```
[36]: df.iloc[[2, 6]]           # rows with index labels 2 and 3
```

```
[36]:   Student_ID  Age Gender  Maths  Science  English  Attendance  Study_Hours  \
2           3    21   Male    74      62      81        94.5         3.5
6           7    21   Male    64      97      72        85.8         1.3

      Sports Result
2      No    Fail
6      No    Fail
```

```
[37]: df.iloc[:, 2]
```

```
[37]: 0      Male
1     Female
2      Male
3      Male
4     Female
...
198    Female
199     Male
200     Male
201     Male
202     Male
Name: Gender, Length: 203, dtype: object
```

```
[38]: df.loc[[4, 8], 'Age':'Attendance']           # label-based slice (INCLUSIVE of 4, 8)
      ↪ 'TP')
```

```
[38]:   Age  Gender  Maths  Science  English  Attendance
4    19  Female    50      98      48        74.9
8    23   Male    51      67      45        61.0
```

```
[39]: df.loc[:, ['Maths', 'English']]           # multiple specific columns
```

```
[39]:   Maths  English
0      81      54
1      89      46
2      74      81
3      86      35
4      50      48
...
198    42      72
199    92      85
200    53      67
201    53      67
```

202 98 43

[203 rows x 2 columns]

```
[40]: # Boolean indexing
male_data = df[df['Gender'] == 'Male']
male_data.head()
```

```
[40]:
```

	Student_ID	Age	Gender	Maths	Science	English	Attendance	Study_Hours	\
0	1	23	Male	81	94	54	66.2	2.7	
2	3	21	Male	74	62	81	94.5	3.5	
3	4	23	Male	86	75	35	97.8	3.0	
5	6	24	Male	47	61	98	70.8	NaN	
6	7	21	Male	64	97	72	85.8	1.3	

```
Sports Result
0    Yes  Fail
2    No  Fail
3    Yes  Pass
5    No  Pass
6    No  Fail
```

```
[41]: # Multiple conditions: use & / | with parentheses (NOT 'and'/'or')
df[(df['Age'] >= 7) & (df['Gender'] == 'Male')].tail()
```

```
[41]:
```

	Student_ID	Age	Gender	Maths	Science	English	Attendance	Study_Hours	\
197	198	20	Male	70	39	100	76.7	4.2	
199	200	19	Male	92	53	85	89.5	1.5	
200	11	19	Male	53	47	67	88.6	3.7	
201	11	19	Male	53	47	67	88.6	3.7	
202	26	22	Male	98	45	43	94.2	5.5	

```
Sports Result
197    No  Pass
199    Yes  Fail
200    Yes  Pass
201    Yes  Pass
202    Yes  Pass
```

```
[42]: # .query() - same filter, more readable for complex conditions
df.query("Age >= 5 and Gender == 'Male']").head(5)
```

```
[42]:
```

	Student_ID	Age	Gender	Maths	Science	English	Attendance	Study_Hours	\
0	1	23	Male	81	94	54	66.2	2.7	
2	3	21	Male	74	62	81	94.5	3.5	
3	4	23	Male	86	75	35	97.8	3.0	
5	6	24	Male	47	61	98	70.8	NaN	

6	7	21	Male	64	97	72	85.8	1.3
---	---	----	------	----	----	----	------	-----

	Sports	Result
0	Yes	Fail
2	No	Fail
3	Yes	Pass
5	No	Pass
6	No	Fail

```
[43]: # .query() - same filter, more readable for complex conditions
df.query("Age >= 6 and Gender == 'Male'").head()
```

```
[43]:   Student_ID  Age  Gender  Maths  Science  English  Attendance  Study_Hours  \
0           1   23   Male    81      94      54        66.2         2.7
2           3   21   Male    74      62      81        94.5         3.5
3           4   23   Male    86      75      35        97.8         3.0
5           6   24   Male    47      61      98        70.8         NaN
6           7   21   Male    64      97      72        85.8         1.3
```

	Sports	Result
0	Yes	Fail
2	No	Fail
3	Yes	Pass
5	No	Pass
6	No	Fail

16 Section 8: Sorting, Ranking, Rename, Drop, Insert, Assign, Replace

```
[44]: # Sort by a single column
df.sort_values(by='Attendance', ascending=False).head()
```

```
[44]:   Student_ID  Age  Gender  Maths  Science  English  Attendance  \
188        189   23   Male    88      85      83        99.9
145        146   23  Female    38      98      81        99.5
66          67   20   Male    78     100     100        99.4
195        196   18   Male    62      79      35        99.4
157        158   18  Female    54      68      41        98.8
```

	Study_Hours	Sports	Result
188	5.8	Yes	Pass
145	4.2	Yes	Pass
66	1.6	Yes	Pass
195	6.6	No	Pass
157	7.6	Yes	Fail

```
[45]: # Sort by multiple columns
df.sort_values(by=['Gender', 'Age'], ascending=[True, False]).head()
```

```
[45]:
```

	Student_ID	Age	Gender	Maths	Science	English	Attendance	Study_Hours	\
14	15	24	Female	57	81	39	69.2	6.5	
18	19	24	Female	92	90	36	92.0	NaN	
46	47	24	Female	35	38	64	85.0	2.6	
56	57	24	Female	67	84	73	95.8	7.0	
83	84	24	Female	58	90	91	83.9	7.1	

	Sports	Result
14	No	Pass
18	Yes	Pass
46	No	Fail
56	No	Pass
83	Yes	Pass

```
[46]: # rank() - assigns a rank to each value (useful for percentile/
      ↪competition-style ranking)
df['Age_rank'] = df['Age'].rank(ascending=False)
df[['Age', 'Age_rank']].head()
```

```
[46]:
```

	Age	Age_rank
0	23	40.0
1	20	115.5
2	21	89.0
3	23	40.0
4	19	143.0

```
[47]: # Rename columns
df_renamed = df.rename(columns={'Maths':'M1','Science':'s1'})
df_renamed.head(2)
```

```
[47]:
```

	Student_ID	Age	Gender	M1	s1	English	Attendance	Study_Hours	Sports	\
0	1	23	Male	81	94	54	66.2	2.7	Yes	
1	2	20	Female	89	84	46	67.3	1.8	No	

	Result	Age_rank
0	Fail	40.0
1	Pass	115.5

```
[48]: # Drop columns
      # Drop one column
df.drop('Age_rank', axis=1, errors='ignore', inplace=True)

      # Drop multiple columns
df.drop(['Maths', 'Science'], axis=1).head(2)
```

```
[48]:
```

	Student_ID	Age	Gender	English	Attendance	Study_Hours	Sports	Result
0	1	23	Male	54	66.2	2.7	Yes	Fail
1	2	20	Female	46	67.3	1.8	No	Pass

```
[49]: # Drop rows
df.drop([0, 1, 2], axis=0).head()
```

```
[49]:
```

	Student_ID	Age	Gender	Maths	Science	English	Attendance	Study_Hours	\
3	4	23	Male	86	75	35	97.8	3.0	
4	5	19	Female	50	98	48	74.9	3.1	
5	6	24	Male	47	61	98	70.8	NaN	
6	7	21	Male	64	97	72	85.8	1.3	
7	8	21	Female	53	51	71	76.3	1.1	

	Sports	Result
3	Yes	Pass
4	No	Pass
5	No	Pass
6	No	Fail
7	Yes	Pass

```
[51]: # Insert a column at a specific position
df = pd.read_csv("student_data.csv") # file

df_copy = df.copy()

df_copy.insert(1, 'AgeGroup',
               np.where(df_copy['Age'] >= 60, 'Senior', 'Adult'))

df_copy.head(3)
```

```
[51]:
```

	Student_ID	AgeGroup	Age	Gender	Maths	Science	English	Attendance	\
0	1	Adult	23	Male	81	94	54	66.2	
1	2	Adult	20	Female	89	84	46	67.3	
2	3	Adult	21	Male	74	62	81	94.5	

	Study_Hours	Sports	Result
0	2.7	Yes	Fail
1	1.8	No	Pass
2	3.5	No	Fail

```
[54]: # assign() - create one or more new columns in a chainable/functional style
df_assigned = df.assign(
    TB_per_DB=lambda x: x['Maths'] / (x['Science'] + 0.02), # +0.01 avoids_
    ↪divide-by-zero
    IsMale=lambda x: x['Gender'] == 'Male' )
df_assigned.head(3)
```

```
[54]:
```

	Student_ID	Age	Gender	Maths	Science	English	Attendance	Study_Hours	\
0	1	23	Male	81	94	54	66.2	2.7	
1	2	20	Female	89	84	46	67.3	1.8	
2	3	21	Male	74	62	81	94.5	3.5	

	Sports	Result	TB_per_DB	IsMale
0	Yes	Fail	0.861519	True
1	No	Pass	1.059272	False
2	No	Fail	1.193163	True

```
[57]: # replace() - swap specific values
df['Gender'].replace({'Male': 'm', })
```

```
[57]:
```

0	m
1	Female
2	m
3	m
4	Female
...	
198	Female
199	m
200	m
201	m
202	m

Name: Gender, Length: 203, dtype: object

17 Section 9: map, apply, DataFrame.map, and Lambda Functions

```
[59]: # map() on a Series with a dictionary
gender_code = {'Male': 2, 'Female': 1}
df['Gender_code'] = df['Gender'].map(gender_code)
df[['Gender', 'Gender_code']].head()
```

```
[59]:
```

	Gender	Gender_code
0	Male	2
1	Female	1
2	Male	2
3	Male	2
4	Female	1

```
[60]: # map() with a function / lambda
df['Age_in_months'] = df['Age'].map(lambda x: x * 14)
df[['Age', 'Age_in_months']].head()
```

```
[60]:   Age  Age_in_months
      0    23           322
      1    20           280
      2    21           294
      3    23           322
      4    19           266
```

```
[62]: # apply() on a Series - equivalent to map() for simple functions
df['M1'] = df['Maths'].apply(lambda x: x ** 4)
df[['Maths', 'M1']].head()
```

```
[62]:   Maths      M1
      0     81  43046721
      1     89  62742241
      2     74  29986576
      3     86  54700816
      4     50   6250000
```

```
[65]: # apply() on a DataFrame, row-wise (axis=1)

def student_risk(row):
    if row['Maths'] < 40 or row['Attendance'] < 75:
        return 'High'
    else:
        return 'Low'

df['Risk'] = df.apply(student_risk, axis=1)

df[['Maths', 'Attendance', 'Risk']].head()
```

```
[65]:   Maths  Attendance  Risk
      0     81         66.2  High
      1     89         67.3  High
      2     74         94.5   Low
      3     86         97.8   Low
      4     50         74.9  High
```

```
[66]: # apply() on a DataFrame, column-wise (axis=0, the default) - function receives
      ↪ a full column
df[['Age', 'Maths', 'Science']].apply(lambda col: col.max() - col.min())
```

```
[66]: Age           7
      Maths        65
      Science      65
      dtype: int64
```

```
[68]: # DataFrame.map() - element-wise transform across the WHOLE frame (numeric,
      ↪ columns shown here)
      df[['Age', 'Maths', 'Science']].map(lambda x: round(x, 1))
```

```
[68]:
```

	Age	Maths	Science
0	23	81	94
1	20	89	84
2	21	74	62
3	23	86	75
4	19	50	98
..	...	...	...
198	21	42	60
199	19	92	53
200	19	53	47
201	19	53	47
202	22	98	45

[203 rows x 3 columns]

```
[70]: # Cleanup helper columns
      df.drop(columns=['Gender_code', 'Age_in_months', 'Risk'], inplace=True,
      ↪ errors='ignore')
      print(df.head())
```

	Student_ID	Age	Gender	Maths	Science	English	Attendance	Study_Hours	\
0	1	23	Male	81	94	54	66.2	2.7	
1	2	20	Female	89	84	46	67.3	1.8	
2	3	21	Male	74	62	81	94.5	3.5	
3	4	23	Male	86	75	35	97.8	3.0	
4	5	19	Female	50	98	48	74.9	3.1	

	Sports	Result
0	Yes	Fail
1	No	Pass
2	No	Fail
3	Yes	Pass
4	No	Pass

18 Section 10: Missing Values

```
[71]: df.isnull().sum() # count of missing values per column
```

```
[71]: Student_ID    0
      Age          0
      Gender       0
      Maths        0
```

```

Science      0
English      0
Attendance   0
Study_Hours  4
Sports       0
Result       0
dtype: int64

```

```
[72]: df.isna().sum()      # isna() is an alias for isnull() - identical behavior
```

```

[72]: Student_ID      0
      Age             0
      Gender          0
      Maths           0
      Science         0
      English         0
      Attendance      0
      Study_Hours     4
      Sports          0
      Result          0
      dtype: int64

```

```
[73]: df.notnull().sum() # inverse: count of NON-missing values per column
```

```

[73]: Student_ID      203
      Age             203
      Gender          203
      Maths           203
      Science         203
      English         203
      Attendance      203
      Study_Hours     199
      Sports          203
      Result          203
      dtype: int64

```

```

[76]: # dropna(): remove rows with ANY missing value
      print("Before:", df.shape)
      print("After dropna():", df.dropna().shape)

```

```

Before: (203, 10)
After dropna(): (199, 10)

```

```
[78]: df['Study_Hours'].fillna(1.0).isnull().sum()
```

```
[78]: np.int64(0)
```

```
[81]: # fillna() with the column mean / median (common imputation strategy)
mean_fill = df['Attendance'].fillna(df['Attendance'].mean())
median_fill = df['Attendance'].fillna(df['Attendance'].median())
print("Mean used:", df['Attendance'].mean())
print("Median used:", df['Attendance'].median())
```

Mean used: 79.928078817734
Median used: 80.8

```
[83]: # fillna() with a GROUPED statistic - more accurate than a single global fill_
      ↪ value
df['Attendance']
df.groupby('Gender')['Attendance'].transform(lambda x: x.fillna(x.mean()))
df['Attendance'].isnull().sum()
```

```
[83]: np.int64(0)
```

```
[84]: # interpolate(): fill gaps using linear (or other) interpolation - useful for_
      ↪ ordered/time-series data
demo = pd.Series([3.0, np.nan, np.nan, 6.0, 7.0])
print(demo.interpolate())
```

```
0    3.0
1    4.0
2    5.0
3    6.0
4    7.0
dtype: float64
```

19 Section 11: Duplicate Data

```
[85]: df.duplicated().sum()          # total number of duplicate rows (keeps first_
      ↪ occurrence as non-duplicate)
```

```
[85]: np.int64(3)
```

```
[86]: df.duplicated(subset=['Age', 'Gender']).sum()          # duplicates based on a_
      ↪ SUBSET of columns
```

```
[86]: np.int64(187)
```

```
[89]: print("Before:", df.shape)
df_clean = df.drop_duplicates()
print("After drop_duplicates():", df_clean.shape)
```

Before: (203, 10)
After drop_duplicates(): (200, 10)

```
[90]: # keep='last' keeps the LAST occurrence instead of the first
df.drop_duplicates(keep='last').shape
```

```
[90]: (200, 10)
```

20 Section 12: String Operations (. str accessor)

```
[91]: df['Gender'].str.lower().head()
```

```
[91]: 0    male
      1   female
      2    male
      3    male
      4   female
      Name: Gender, dtype: object
```

```
[100]: import pandas as pd

# strip() removes leading/trailing whitespace

messy = pd.Series([' Male', 'Female', ' Male '])

print(messy.str.strip())
```

```
0    Male
1  Female
2    Male
dtype: object
```

```
[101]: df['Gender'].str.replace('Male', 'M', regex=False).head()
```

```
[101]: 0    M
      1  Female
      2    M
      3    M
      4  Female
      Name: Gender, dtype: object
```

```
[103]: # Split() - break a string column into parts
names = pd.Series(['Anjali Chopade', 'Shravani Kamble', 'Pratiksha Khot'])
names.str.split(' ')
```

```
[103]: 0    [Anjali, Chopade]
      1  [Shravani, Kamble]
      2    [Pratiksha, Khot]
      dtype: object
```

```
[104]: names.str.split(' ', expand=True) # expand=True returns a
↳ DataFrame of split parts
```

```
[104]:      0      1
0   Anjali Chopade
1   Shravani Kamble
2  Pratiksha Khot
```

```
[106]: # contains() - boolean mask for substring matching (supports regex)
df[df['Gender'].str.contains('Fem')].head(3)
```

```
[106]:   Student_ID  Age  Gender  Maths  Science  English  Attendance  Study_Hours  \
1           2   20  Female     89         84         46          67.3         1.8
4           5   19  Female     50         98         48          74.9         3.1
7           8   21  Female     53         51         71          76.3         1.1

      Sports Result
1       No   Pass
4       No   Pass
7      Yes   Pass
```

```
[108]: # extract() - pull out a pattern using a regex capture group
codes = pd.Series(['ID-001', 'ID-002', 'ID-103'])
codes.str.extract(r'ID-(\d+)')
```

```
[108]:      0
0   001
1   002
2   103
```

21 Section 13: Date and Time

```
[110]: # to_datetime() converts strings/objects into proper datetime64 dtype
dates = pd.to_datetime(['2026-01-19', '2026-02-4', '2026-03-07'])
print(dates)
print(dates.dtype)
```

```
DatetimeIndex(['2026-01-19', '2026-02-04', '2026-03-07'],
dtype='datetime64[ns]', freq=None)
datetime64[ns]
```

```
[113]: # Attach a synthetic 'AdmissionDate' column
rng = np.random.default_rng(7)

df['AdmissionDate'] = (
    pd.to_datetime('2026-01-06') +
    pd.to_timedelta(rng.integers(0, 100, len(df)), unit='D')
```

```
)

# Display the first 5 rows
print(df[['Student_ID', 'Age', 'AdmissionDate']].head())
```

	Student_ID	Age	AdmissionDate
0	1	23	2026-04-10
1	2	20	2026-03-09
2	3	21	2026-03-15
3	4	23	2026-04-05
4	5	19	2026-03-04

```
[130]: # Extract month and day from Admission Date
```

```
df['AdmitMonth'] = df['AdmissionDate'].dt.month
df['AdmitDay'] = df['AdmissionDate'].dt.day_name()

print(df[['AdmissionDate', 'AdmitMonth', 'AdmitDay']].head())
```

	AdmissionDate	AdmitMonth	AdmitDay
0	2026-04-10	4	Friday
1	2026-03-09	3	Monday
2	2026-03-15	3	Sunday
3	2026-04-05	4	Sunday
4	2026-03-04	3	Wednesday

```
[131]: # date_range() - generate a sequence of dates
```

```
pd.date_range(start='2026-01-06', periods=8, freq='D')
```

```
[131]: DatetimeIndex(['2026-01-06', '2026-01-07', '2026-01-08', '2026-01-09',
                    '2026-01-10', '2026-01-11', '2026-01-12', '2026-01-13'],
                    dtype='datetime64[ns]', freq='D')
```

```
[134]: pd.date_range(start='2026-01-10', periods=5, freq='ME')           # month-end
        ↪ frequency
```

```
[134]: DatetimeIndex(['2026-01-31', '2026-02-28', '2026-03-31', '2026-04-30',
                    '2026-05-31'],
                    dtype='datetime64[ns]', freq='ME')
```

```
[137]: rng = np.random.default_rng(7)
```

```
df['AdmitDate'] = (
    pd.to_datetime('2026-01-06') +
    pd.to_timedelta(rng.integers(0, 100, len(df)), unit='D')
)

print(df[['AdmitDate']].head())
```

```

    AdmitDate
0 2026-04-10
1 2026-03-09
2 2026-03-15
3 2026-04-05
4 2026-03-04

```

22 Section 14: GroupBy — Split, Apply, Combine

```

[143]: # Split + apply (mean) + combine, in one line
df.groupby('Gender')['Age'].mean()

```

```

[143]: Gender
Female    20.468085
Male      20.623853
Name: Age, dtype: float64

```

```

[145]: # size() - count of rows per group (includes NaN counts unlike count())
df.groupby('Gender').size()

```

```

[145]: Gender
Female      94
Male       109
dtype: int64

```

```

[146]: # get_group() - pull out a single group as its own DataFrame
df.groupby('Gender').get_group('Female').head()

```

```

[146]:
   Student_ID  Age  Gender  Maths  Science  English  Attendance  Study_Hours  \
1           2   20  Female     89      84      46         67.3         1.8
4           5   19  Female     50      98      48         74.9         3.1
7           8   21  Female     53      51      71         76.3         1.1
12          13   19  Female     89      69      84         61.1         2.5
13          14   19  Female     96      40      44         68.9         7.6

```

```

   Sports  Result  AdmissionDate  AdmitMonth  AdmitDay  AdmitDate
1     No    Pass    2026-03-09           3    Monday  2026-03-09
4     No    Pass    2026-03-04           3  Wednesday  2026-03-04
7    Yes    Pass    2026-01-28           1  Wednesday  2026-01-28
12   Yes    Pass    2026-04-07           4    Tuesday  2026-04-07
13   No    Pass    2026-01-06           1    Tuesday  2026-01-06

```

```

[147]: # agg() - apply multiple aggregation functions at once
df.groupby('Gender')['Age'].agg(['mean', 'min', 'max', 'count'])

```

```

[147]:
      mean  min  max  count
Gender

```

Female	20.468085	17	24	94
Male	20.623853	17	24	109

```
[150]: # agg() with different functions per column df.groupby('Gender').agg({
summary = df.groupby('Gender').agg({
    'Age': 'mean',
    'Maths': ['mean', 'max'],
    'Result': lambda x: (x == 'Pass').sum()
})

print(summary)
```

	Age	Maths	Result
	mean	mean max	<lambda>
Gender			
Female	20.468085	66.031915	99 82
Male	20.623853	66.862385	100 95

```
[155]: # transform() - returns an output the SAME SHAPE as the input (broadcast back
↳ to every row)
df['Age']
df.groupby('Gender')['Age'].transform('mean')
df[['Gender', 'Age', 'Age']].head()
```

```
[155]:   Gender  Age  Age
0   Male   23   23
1  Female   20   20
2   Male   21   21
3   Male   23   23
4  Female   19   19
```

```
[157]: # filter() - keep only groups that satisfy a condition
df.groupby('Gender').filter(lambda g: g['Age'].mean() > 4).head()
```

```
[157]:   Student_ID  Age  Gender  Maths  Science  English  Attendance  Study_Hours  \
0           1   23   Male    81      94      54         66.2         2.7
1           2   20  Female    89      84      46         67.3         1.8
2           3   21   Male    74      62      81         94.5         3.5
3           4   23   Male    86      75      35         97.8         3.0
4           5   19  Female    50      98      48         74.9         3.1
```

	Sports	Result	AdmissionDate	AdmitMonth	AdmitDay	AdmitDate
0	Yes	Fail	2026-04-10	4	Friday	2026-04-10
1	No	Pass	2026-03-09	3	Monday	2026-03-09
2	No	Fail	2026-03-15	3	Sunday	2026-03-15
3	Yes	Pass	2026-04-05	4	Sunday	2026-04-05
4	No	Pass	2026-03-04	3	Wednesday	2026-03-04

23 Section 15: Aggregation Functions

```
[169]: print("mean: Not Applicable")
       print("median: Not Applicable")
       print("mode:\n", df['Gender'].mode())
```

```
mean: Not Applicable
median: Not Applicable
mode:
0    Male
Name: Gender, dtype: object
```

```
[171]: print(df.columns)
```

```
Index(['Student_ID', 'Gender', 'Maths', 'Science', 'English', 'Attendance',
       'Study_Hours', 'Sports', 'Result', 'AdmissionDate', 'AdmitMonth',
       'AdmitDay', 'AdmitDate'],
      dtype='object')
```

```
[172]: print("std:", df['Maths'].std())
       print("var:", df['Maths'].var())
       print("count:", df['Maths'].count())
       print("nunique:", df['Gender'].nunique())
```

```
std: 19.036006820622315
var: 362.3695556747793
count: 203
nunique: 2
```

```
[173]: print("min:", df['Gender'].min())
       print("max:", df['Gender'].max())
```

```
min: Female
max: Male
```

```
[174]: df['Science'].nlargest()           # top 5 values by default
```

```
[174]: 16    100
       66    100
       89    100
       185   100
       191   100
       Name: Science, dtype: int64
```

```
[175]: df['Maths'].nsmallest(3) # smallest 3 values
```

```
[175]: 17    35
       19    35
       22    35
```

Name: Maths, dtype: int64

```
[176]: df['Maths'].product()           # product of all values (can underflow/overflow,
      ↪ for many values - use cautiously)
```

```
[176]: np.int64(0)
```

```
[177]: # Value_counts() - frequency count per category (closely related,
      ↪ to aggregation)
      df['Gender'].value_counts()
```

```
[177]: Gender
      Male      109
      Female    94
      Name: count, dtype: int64
```

```
[178]: df['Gender'].value_counts(normalize=True)           # as proportions instead of,
      ↪ raw [un]: counts
```

```
[178]: Gender
      Male      0.536946
      Female    0.463054
      Name: proportion, dtype: float64
```

```
[179]: # Combine several aggregations into a single summary table for the whole,
      ↪ DataFrame
      df.select_dtypes(include=np.number).agg(['mean', 'std', 'min', 'max']).T
```

```
[179]:
```

	mean	std	min	max
Student_ID	99.251232	58.356373	1.0	200.0
Maths	66.477833	19.036007	35.0	100.0
Science	67.339901	20.107575	35.0	100.0
English	67.251232	18.816228	35.0	100.0
Attendance	79.928079	11.853138	60.4	99.9
Study_Hours	4.621608	2.086921	1.0	8.0
AdmitMonth	2.399015	1.030908	1.0	4.0

```
[ ]:
```